CryptoVault : rapport d'analyse de sécurité et d'audit cryptographique

Projet de cryptographie moderne · Messagerie chiffrée de bout en bout et coffre-fort de fichiers

Auteur : [Prénom Nom] · Octobre 2026

Résumé exécutif

CryptoVault est un prototype de système d'information sécurisé qui combine deux services : une **messagerie chiffrée de bout en bout**, fondée sur un handshake éphémère de type TLS 1.3, et un **coffre-fort de fichiers multi-destinataires**, dont l'intégrité est vérifiable bloc par bloc grâce à un arbre de Merkle.

Les briques mathématiques et asymétriques ont été réimplémentées à partir de zéro, sans bibliothèque cryptographique externe : arithmétique modulaire, tests de primalité, RSA, Diffie–Hellman, courbes elliptiques de Weierstrass sur $\mathbb{F}_p$, PRNG HMAC-DRBG et PKI X.509. Seules les primitives symétriques finales (SHA-256 et AES-256) s'appuient sur des implémentations de référence.

Ce rapport présente :

1. l'architecture du système et ses choix de conception ;
2. la traçabilité entre fonctionnalités, modules logiciels et chapitres du cours ;
3. le modèle de menaces (Dolev-Yao, STRIDE) ;
4. les attaques de laboratoire, avec leurs résultats mesurés et la contre-mesure retenue pour chacune ;
5. la validation par 65 tests unitaires et d'intégration ;
6. les limites du prototype (canaux auxiliaires, mémoire, absence d'audit indépendant) ;
7. l'impact de la menace quantique et une feuille de route de migration vers les standards NIST post-quantiques.

1. Architecture générale

1.1 Organisation du projet

Le projet compte neuf modules : huit bibliothèques cryptographiques regroupées dans `cryptolib/` et une application intégratrice dans `app/`. S'y ajoutent un laboratoire d'attaques (`lab/`), la suite de tests (`tests/`) et la documentation (`report/`).

```
cryptovault/
├── cryptolib/
|   ├── hashing.py     # M1 : SHA-256, HMAC (RFC 2104), PBKDF2, arbre de
Merkle
|   ├── numtheory.py   # M2 : Euclide étendu, CRT, exponentiation rapide,
Miller-Rabin
|   ├── rsa.py         # M3 : RSA-OAEP (SHA-256), accélération CRT,
signatures
|   ├── dh.py          # M4 : DH MODP-2048 (RFC 3526), HKDF (RFC 5869), DH
authentifié
|   ├── ecc.py         # M5 : NIST P-256, ECDH, ECDSA déterministe (RFC 6979)
|   ├── symmetric.py   # M6 : AES (GCM, CBC, CTR, ECB), PKCS#7, Vigenère,
Feistel
|   ├── pki.py         # M7 : X.509, Root CA / Sub-CA, CRL, KeyStore, Shamir
|   └── prng.py        # M8 : LCG, MT19937, HMAC_DRBG (NIST SP 800-90A)
├── app/               # M9 : application intégrée
|   ├── server.py      # passerelle, autorité de certification, annuaire,
relais
|   ├── client.py      # client terminal, KeyStore chiffré, handshake, anti-
rejeu
|   ├── vault.py       # coffre-fort : enveloppe hybride, Merkle, signatures
|   └── gui.py         # interface web (SPA à 5 onglets)
├── lab/               # scripts d'attaques pédagogiques et benchmarks
├── tests/             # 65 tests unitaires et d'intégration
└── report/            # documentation, captures, rapport
```

1.2 Scénario A : messagerie chiffrée de bout en bout

Enregistrement. Alice génère localement sa paire P-256 et envoie sa clé publique avec une demande de certificat. Le serveur stocke son mot de passe sous forme dérivée (sel aléatoire de 128 bits, PBKDF2-HMAC-SHA256, 100 000 itérations) et lui délivre un certificat X.509 signé par l'autorité intermédiaire. La clé privée reste dans un KeyStore local chiffré (AES-256-GCM, clé dérivée du mot de passe par PBKDF2).

Handshake de session.

1. Le client génère une paire éphémère ECDHE P-256, signe sa clé publique éphémère avec sa clé d'identité à long terme, puis envoie son certificat, la clé éphémère et la signature.
2. Le serveur valide la chaîne de certification jusqu'à la Root CA, vérifie la signature, génère sa propre paire éphémère qu'il signe à son tour, puis calcule le secret ECDH.
3. Le client effectue les vérifications symétriques.
4. Chaque côté dérive la clé de session par HKDF (RFC 5869) à partir du secret ECDH.

Échange de messages. Chaque trame est chiffrée en AES-256-GCM avec un nonce aléatoire de 96 bits. Elle transporte un numéro de séquence strictement croissant, placé en données associées (AAD). Le récepteur rejette toute trame dont le numéro est inférieur ou égal au dernier reçu.

Propriété	Mécanisme
Confidentialité persistante (PFS)	Secrets ECDHE éphémères, détruits après la session : la compromission ultérieure de la clé d'identité ne permet pas de déchiffrer les échanges passés
Authentification mutuelle	Clés éphémères signées par des clés certifiées X.509
Intégrité et authenticité	AEAD AES-256-GCM (étiquette GHASH de 128 bits)
Anti-rejeu	Numéro de séquence monotone vérifié à la réception

1.3 Scénario B : coffre-fort de fichiers

Le coffre-fort applique le chiffrement hybride par enveloppe (KEM/DEM), complété d'une signature d'auteur et d'un arbre de Merkle :

1. le fichier est découpé en blocs de 1 Ko et un arbre de Merkle donne la racine H_root ;
2. l'auteur signe la racine (ECDSA P-256 ou RSA) ;
3. une clé de fichier K_file de 256 bits est tirée du CSPRNG HMAC_DRBG, puis le contenu est chiffré en AES-256-GCM ;
4. K_file est chiffrée séparément pour chaque destinataire autorisé (RSA-OAEP ou ECIES/ECDH avec sa clé publique).

Commandes disponibles :

- `vault add` : découpe, construit l'arbre, signe, chiffre et crée l'enveloppe du propriétaire.
- `vault share` : le propriétaire déchiffre K_file avec sa clé privée, puis la rechiffre pour un nouvel utilisateur inscrit dans la PKI.
- `vault get` : déchiffre K_file puis le contenu, recalcule l'arbre de Merkle, vérifie la signature de l'auteur et sa chaîne de certification jusqu'à la Root CA.
- `vault list` : affiche les fichiers accessibles à l'utilisateur, leur taille, leur auteur, leurs destinataires, leur racine de Merkle et l'état de la signature.

Le contrôle d'accès repose sur la possession d'une enveloppe : un utilisateur ne peut lister et déchiffrer que les fichiers pour lesquels une enveloppe de clé a été créée à son nom.

1.4 Interface web

L'interface est une application monopage (`gui.py`) en cinq onglets :

1. **Tableau de bord et PKI** : utilisateurs, état des certificats, révocation (CRL), suppression de compte.
2. **Messagerie E2EE** : déverrouillage du KeyStore, handshake ECDHE, flux chiffré, test de rejeu.
3. **Coffre-fort** : dépôt et signature de fichiers, téléchargement vérifié par la racine de Merkle, partage, suppression par le propriétaire.
4. **Laboratoire d'attaques** : exécution interactive des démonstrations.
5. **Terminal CLI** : émulateur permettant de lancer les commandes `cryptovault` depuis le navigateur.

[Figures 1 à 3 : captures du tableau de bord, de la messagerie E2EE et du coffre-fort (formulaire de dépôt, puis partage et test de lecture)]

2. Traçabilité : fonctionnalités, modules et chapitres du cours

Fonctionnalité	Modules	Chapitres	Mécanismes
Stockage des mots de passe	M1, M8, M9	Hachage, PRNG	PBKDF2-HMAC-SHA256, sel de 128 bits issu du CSPRNG, 100 000 itérations
Génération des clés RSA et P-256	M2, M3, M5	Arithmétique, RSA, courbes elliptiques	Miller-Rabin (k = 40), Euclide étendu ($e \cdot d \equiv 1 \bmod \varphi(n)$), courbe $y^2 = x^3 - 3x + b$ sur $\mathbb{F}_p$
Certificats X.509	M7, M9	PKI	Root CA auto-signée → Sub-CA → certificats feuilles, extensions is_ca et key_usage, CRL
Protection des clés privées (KeyStore)	M1, M6, M7, M9	PKI, symétrique, KDF	Conteneur AES-256-GCM, clé dérivée par PBKDF2, rotation du mot de passe

Fonctionnalité	Modules	Chapitres	Mécanismes
Handshake de session	M4, M5, M9	DH, ECDH, PKI	ECDHE P-256, signatures ECDSA croisées, HKDF
Canal de messagerie et anti-rejeu	M6, M9	Symétrique, intégrité	AES-256-GCM, nonces de 96 bits, séquence monotone
Intégrité granulaire des fichiers	M1, M9	Hachage, Merkle	Feuilles H(0x00 ‖ bloc), nœuds H(0x01 ‖ gauche ‖ droite), preuves d'inclusion en O(log N)
Enveloppe multi-destinataires	M3, M5, M6, M9	RSA, ECC, symétrique	K_file aléatoire de 256 bits, AES-GCM, RSA-OAEP ou ECIES
Signature d'auteur	M3, M5, M7, M9	Signatures, chaînes de confiance	ECDSA (RFC 6979) ou RSA-PSS, vérification de chaîne à `vault get`
Protection de la clé racine	M7	Gestion des clés	Partage de Shamir (3, 5) sur $\mathbb{F}_p$, reconstruction par interpolation de Lagrange

Points techniques par module

M1, hachage. HMAC est implémenté sans le module `hmac` standard : HMAC(K, M) = SHA-256((K′ ⊕ opad) ‖ SHA-256((K′ ⊕ ipad) ‖ M)), avec ipad = 0x36 et opad = 0x5C répétés sur 64 octets. La vérification se fait en temps constant par XOR cumulatif, sans sortie anticipée. L'arbre de Merkle utilise un préfixe de domaine différent pour les feuilles et les nœuds internes.

M2, arithmétique. L'identité de Bézout ($a \cdot u + b \cdot v = \text{pgcd}(a, b)$) donne l'inverse modulaire. L'exponentiation square-and-multiply ramène le coût de O(exp) à O(log exp). Miller-Rabin écrit $n - 1 = 2^s \cdot d$ et teste des témoins aléatoires ; avec k = 40 rondes, la probabilité qu'un composé passe le test est inférieure à $4^{-40} \approx 8{,}3 \cdot 10^{-25}$.

M3, RSA. Les clés utilisent e = 65537. Le déchiffrement passe par le CRT (d_p, d_q, q_inv), ce qui divise le coût d'un facteur proche de 4 ; le benchmark mesure environ ×3,8. Le chiffrement utilise RSA-OAEP (RFC 8017, MGF1-SHA256), qui supprime la malléabilité du RSA brut.

M4, Diffie–Hellman. Le groupe est le MODP-2048 de la RFC 3526 ($g = 2$). Le secret brut passe par HKDF : Extract ($PRK = HMAC(sel, IKM)$) puis Expand pour obtenir des clés indépendantes. Les contributions éphémères sont signées pour empêcher l'attaque de l'homme du milieu.

M5, courbes elliptiques. La courbe est NIST P-256 ($a = -3$). L'addition utilise la pente $\lambda = (y_q - y_p)/(x_q - x_p)$ et le doublement $\lambda = (3x_p^2 + a)/(2y_p)$, avec une multiplication scalaire double-and-add. Le nonce ECDSA est dérivé de façon déterministe par HMAC-SHA256 (RFC 6979).

M6, symétrique. AES est disponible en modes ECB, CBC, CTR et GCM, avec bourrage PKCS#7. GCM est le mode par défaut : CTR assure la confidentialité et GHASH sur $\mathbb{F}_{2^{128}}$ produit une étiquette de 16 octets.

M7, PKI. `verify_certificate_chain` contrôle la validité temporelle, la signature de l'émetteur, les contraintes de base (CA:TRUE/FALSE, keyCertSign) et l'absence de révocation sur la CRL. Le KeyStore permet la rotation du mot de passe sans écrire la clé privée en clair sur le disque.

M8, PRNG. Trois générateurs sont comparés : LCG (prédictible après trois sorties), MT19937 (période $2^{19937} - 1$ mais état clonable après 624 sorties) et HMAC_DRBG, source unique d'aléa pour les clés, nonces, IV et sels de l'application.

3. Modèle de menaces

L'évaluation suit le modèle de Dolev-Yao : l'adversaire contrôle le réseau, mais ne casse pas les primitives. Elle est complétée par une lecture STRIDE.

Attaquant	Capacités	Objectif	Contre-mesure
Passif (Eve)	Écoute tout le trafic réseau, sans le modifier	Lire les messages et les fichiers, retrouver des clés	ECDHE P-256 (≈ 128 bits de sécurité), AES-256-GCM, PFS
Actif (Mallory, MITM)	Intercepte, modifie, injecte, supprime, rejoue des trames	Usurper une identité, altérer les données, imposer ses clés	Authentification mutuelle par certificats, signatures des clés éphémères, étiquette GCM, séquence anti-rejeu

Attaquant	Capacités	Objectif	Contre-mesure
Serveur compromis ou curieux	Accès root, lecture de la base et des fichiers stockés	Lire les mots de passe, déchiffrer le coffre	PBKDF2 avec sel, enveloppes chiffrées pour chaque utilisateur : le serveur ne détient jamais K_file en clair ni les clés privées
Vol de terminal	Copie du disque et du fichier KeyStore	Extraire la clé privée pour usurper l'utilisateur	KeyStore chiffré en AES-256-GCM, clé dérivée du mot de passe par PBKDF2

Pour le vol de terminal, la sécurité dépend de la **force du mot de passe** : PBKDF2 ralentit chaque essai, mais un mot de passe faible reste vulnérable à une attaque par dictionnaire hors ligne.

Hors périmètre : compromission du poste pendant une session active, attaques par canaux auxiliaires (section 6), compromission de la clé racine hors du schéma de Shamir.

4. Attaques de laboratoire

Chaque script du dossier `lab/` démontre une vulnérabilité sur des paramètres volontairement faibles, puis la contre-mesure retenue dans CryptoVault. Six démonstrations sont rejouables depuis l'onglet « Laboratoire d'attaques » : anniversaires, Carmichael, failles RSA, MITM de Mallory, nonce ECDSA, fuite ECB. Les durées indiquées dépendent de la machine d'exécution.

[Figure 4 : capture du laboratoire d'attaques]

4.1 Collision d'anniversaire sur un hachage tronqué (`attack_collision_birthday.py`, M1)

- **Principe.** Pour une empreinte de n bits, une collision apparaît avec une probabilité de 50 % après environ $1{,}177 \cdot 2^{n/2}$ tirages.
- **Résultat.** Sur SHA-256 tronqué à 32 bits, la théorie prévoit environ 77 000 essais. Une collision a été trouvée en 62 984 essais, en 0,08 s.
- **Contre-mesure.** Empreinte SHA-256 complète : le coût d'une collision monte à 2^{128}.

4.2 Nombres de Carmichael (`benchmark_numtheory.py`, M2)

- **Principe.** Les nombres de Carmichael (561, 1105, 1729) vérifient $a^{n-1} \equiv 1 \pmod{n}$ pour toute base a première avec n, et trompent donc le test de Fermat.

- **Résultat.** Le test de Fermat déclare 561 premier à chaque essai. Miller-Rabin le déclare composé dès le premier témoin.
- **Contre-mesure.** Miller-Rabin avec k = 40 rondes.

4.3 Quatre attaques sur RSA (`attack_rsa_lab.py`, M3)

Attaque	Principe	Résultat	Contre-mesure
Rho de Pollard	Détection de cycle dans $x_{i+1} = x_i^2 + 1 \bmod n$, coût $O(n^{1/4})$	Module de 64 bits (n = 12711679080277320491) factorisé en 0,004 s : p = 2772592553, q = 4584762517	Modules de 2048 bits ou plus
Malléabilité (RSA brut)	$c \cdot 2^e \bmod n = (2m)^e \bmod n$	Mallory double un montant chiffré sans connaître d	RSA-OAEP avec MGF1-SHA256
Petit exposant $e = 3$ sans bourrage	Si $m^3 < n$, alors $c = m^3$ dans $\mathbb{Z}$	Racine cubique entière, 0,0001 s	e = 65537 et OAEP
Facteurs partagés	$\gcd(n_1, n_2) = p$ si l'aléa est défaillant	Factorisation simultanée en 0,0002 s	CSPRNG HMAC_DRBG

4.4 Diffie–Hellman : MITM et BSGS (`attack_dh_mitm.py`, `attack_dh_bsgs.py`, M4)

- **Homme du milieu.** Sans authentification, Mallory substitue ses propres clés publiques et établit deux secrets distincts, l'un avec Alice, l'autre avec Bob. Elle lit puis falsifie le message (« Virement de 1 000 000 € vers le compte secret » devient « … vers le compte de MALLORY »), et Bob l'accepte. **Contre-mesure :** DH authentifié (Station-to-Station) : chaque contribution éphémère est signée avec une clé certifiée par la PKI.
- **Baby-Step Giant-Step.** Avec $m = \lceil \sqrt{p} \rceil$ et $x = i \cdot m - j$, l'algorithme résout $h = g^x \bmod p$ en temps et mémoire $O(\sqrt{p})$. Un exposant de 28 bits est retrouvé en 0,18 s, un exposant de 36 bits en 1,85 s. **Contre-mesure :** groupe MODP-2048, pour lequel BSGS coûterait environ 2^{1024} opérations ; le crible algébrique reste hors de portée (≥ 112 bits de sécurité classique).

4.5 Réutilisation du nonce ECDSA (`attack_ecc_nonce.py`, M5)

- **Principe.** Avec $s = k^{-1}(z + r \cdot d) \bmod n$, deux signatures partageant le même k donnent $k = (z_1 - z_2)/(s_1 - s_2) \bmod n$, puis $d = (s_1 \cdot k - z_1)/r \bmod n$.
- **Résultat.** Sur P-256, la clé privée est retrouvée en 0,0001 s à partir de deux signatures. C'est la faille exploitée contre la PlayStation 3.

- **Contre-mesure.** Nonce déterministe RFC 6979, dérivé par HMAC-SHA256 de la clé privée et du haché du message.

4.6 Faiblesses symétriques (`attack_symmetric_lab.py`, M6)

- **Fuite visuelle d'ECB.** Des blocs clairs identiques donnent des blocs chiffrés identiques : sur une image BMP, la silhouette du motif reste visible après chiffrement ECB, alors que CBC et GCM produisent un bruit sans corrélation.
- **Bit-flipping sur CBC.** Avec $c'_{i-1} = c_{i-1} \oplus \Delta$, on obtient $p'_i = p_i \oplus \Delta$. Le texte `role=user;access=read` est transformé en `role=admin;access=root` et le serveur accorde les droits. En GCM, la modification d'un seul bit invalide l'étiquette et le message est rejeté.
- **Réutilisation d'un masque jetable.** $c_1 \oplus c_2 = m_1 \oplus m_2$: les deux clairs sont reconstruits par analyse de mots probables, sans connaître la clé.
- **Contre-mesure commune.** Abandon d'ECB et de CBC non authentifié ; AES-256-GCM avec un nonce de 96 bits régénéré à chaque appel.

4.7 Attaques sur la PKI (`demo_pki_lab.py`, M7)

1. **Certificat altéré** (champ Subject modifié) : rejeté à la vérification de la signature de l'émetteur.
2. **Certificat expiré** : rejeté par le contrôle de validité temporelle.
3. **Certificat révoqué** : rejeté après inscription du numéro de série dans la CRL signée.
4. **Usurpation de pouvoir CA** : une entité feuille (is_ca = False) qui émet un certificat est rejetée par `verify_certificate_chain`.
5. **Protection de la clé racine** : avec le partage de Shamir (3, 5), deux parts ne révèlent rien et trois parts reconstruisent exactement la clé.

4.8 PRNG non cryptographiques (`attack_prng_lab.py`, M8)

- **LCG.** Avec $X_{n+1} = a \cdot X_n + c \bmod m$, trois états consécutifs donnent $a = (X_2 - X_1)(X_1 - X_0)^{-1} \bmod m$, puis c : toutes les sorties futures sont prédites (0,0001 s).
- **MT19937.** La fonction de sortie (« tempering ») est une suite de quatre opérations inversibles. À partir de 624 sorties de 32 bits, `untemper()` reconstruit l'état interne : les 1 000 valeurs suivantes du générateur Python et du clone sont identiques.
- **Contre-mesure.** HMAC_DRBG (NIST SP 800-90A), ré-ensemencé depuis l'entropie du système d'exploitation.

5. Validation expérimentale

La suite de tests, fondée sur `unittest` et sur des vecteurs de référence (RFC 4231, RFC 5869, RFC 8017, NIST), compte 65 tests et passe intégralement.

Fichier	Périmètre	Tests
`test_hashing.py`	HMAC (RFC 4231), avalanche, Merkle, PBKDF2	8
`test_numtheory.py`	Euclide étendu, CRT, Miller-Rabin (Carmichael 561, 1105), exponentiation	9
`test_rsa.py`	Clés 1024/2048, OAEP, signatures PSS, accélération CRT	9
`test_dh.py`	MODP-2048, HKDF, signatures DH	6
`test_ecc.py`	Addition/doublement, P-256, ECDH, nonce RFC 6979, ECDSA	8
`test_symmetric.py`	César, Vigenère, OTP, PKCS#7, AES (GCM/CBC/CTR/ECB), Feistel	9
`test_pki.py`	Chaîne X.509, Root CA, Sub-CA, CRL, Shamir (3, 5)	7
`test_prng.py`	LCG, MT19937, HMAC_DRBG, tests statistiques	5
`test_vault.py`	Coffre hybride multi-propriétaires, Merkle, partage, révocation	4
Total		**65**

```
python3 -m unittest discover -s cryptovault/tests -p "test_*.py" -v
```

Résultat : `Ran 65 tests … OK`.

[Figure 5 : capture du terminal avec les 65 tests]

Un test réussi atteste la conformité fonctionnelle aux vecteurs de référence, pas l'absence de faille d'implémentation (voir section 6).

6. Limites du prototype

6.1 « Never roll your own crypto »

1. **Correction fonctionnelle ne veut pas dire sécurité.** Un code peut réussir tous ses tests et rester vulnérable à des attaques passives ou physiques.

2. **Les cas limites sont silencieux.** Accepter une clé publique de sous-groupe d'ordre faible en Diffie-Hellman ou réutiliser un nonce k n'émet aucune erreur et ruine pourtant la sécurité.
3. **L'audit a un coût.** OpenSSL, BoringSSL ou libsodium cumulent des années d'audits, de vérifications formelles et de retours sur incidents.

6.2 Canaux auxiliaires

- **Timing sur l'exponentiation modulaire.** Dans `exp_mod`, la multiplication n'est exécutée que si le bit de l'exposant vaut 1 : le temps dépend du poids de Hamming de l'exposant secret (d en RSA, a en DH), ce qui permet à un attaquant de reconstituer la clé bit à bit. *Correction industrielle :* échelle de Montgomery en temps constant, ou aveuglement (calculer $(m \cdot r^e)^d \cdot r^{-1} \bmod n$).
- **Branchements en ECC.** L'addition de points distingue P = Q (tangente) et P ≠ Q (sécante), ce qui crée des variations de temps et d'accès au cache exploitables par Flush+Reload. *Correction :* coordonnées projectives complètes ou courbes d'Edwards (Ed25519) avec formules unifiées.
- **Consommation et matériel.** Python n'offre aucune garantie de temps constant ni d'instructions dédiées (AES-NI, extensions ARM), contrairement aux bibliothèques certifiées.

6.3 Gestion de la mémoire

En Python, le ramasse-miettes laisse les octets d'une clé en mémoire jusqu'à un passage imprévisible, et peut les déplacer. Un dump mémoire ou une attaque Cold Boot peut donc en extraire d'anciens secrets. *Correction industrielle :* `mlock`, puis effacement explicite (`OPENSSL_cleanse`, `memset_s`).

6.4 Autres limites

- La robustesse du KeyStore dépend de la qualité du mot de passe choisi.
- Le prototype n'a fait l'objet d'aucun audit indépendant.

7. Menace quantique et migration

Primitive	Algorithme quantique	Impact
RSA-2048 (M3)	Shor	Rupture totale (temps polynomial)

Primitive	Algorithme quantique	Impact
Diffie-Hellman MODP (M4)	Shor	Rupture totale (logarithme discret)
ECDH / ECDSA P-256 (M5)	Shor	Rupture totale (ECDLP)
AES-256-GCM (M6)	Grover	Sécurité effective ramenée à 128 bits : reste sûr
SHA-256 (M1)	Grover, BHT	Recherche de collisions en $O(2^{n/3})$, soit environ 2^{85} : suffisant

Asymétrique. L'algorithme de Shor résout factorisation et logarithme discret (groupes finis et courbes elliptiques) en temps polynomial. Toutes les clés asymétriques de CryptoVault deviendraient calculables sur un ordinateur quantique tolérant aux pannes de taille suffisante, dont le nombre de qubits requis fait l'objet d'estimations qui baissent régulièrement. Le risque « récolter maintenant, déchiffrer plus tard » concerne déjà les données à longue durée de vie.

Symétrique. Grover donne un gain quadratique : AES-128 tomberait à 2^{64} opérations, mais AES-256 conserve 128 bits de sécurité effective.

Feuille de route (standards NIST d'août 2024) :

1. **Échange de clés : ML-KEM (FIPS 203, ex-Kyber).** Remplacer ECDHE par ML-KEM-768 ou ML-KEM-1024. Phase transitoire hybride : K_session = HKDF(secret_ECDH ‖ secret_ML-KEM), sûre tant que l'un des deux algorithmes tient.
2. **Signatures et PKI : ML-DSA (FIPS 204, ex-Dilithium).** Remplacer ECDSA et RSA pour les certificats X.509, avec SLH-DSA (FIPS 205, ex-SPHINCS+), fondée uniquement sur le hachage, comme ancre de confiance à très long terme.
3. **Symétrique et hachage.** Conserver AES-256 et SHA-256 ou SHA-384.

8. Briques complémentaires

1. **Partage de secret de Shamir (k, n) = (3, 5).** Il protège la clé privée de la Root CA : trois officiers sur cinq la reconstruisent par interpolation de Lagrange, et deux parts ne donnent aucune information (sécurité inconditionnelle).
2. **Cryptanalyse automatique de Vigenère.** La longueur de clé est estimée par l'indice de coïncidence de Friedman, puis chaque lettre par minimisation du χ^2 sur les fréquences du français.

3. **Réseau de Feistel à 4 rondes.** Il illustre l'inversibilité structurelle sans que la fonction de ronde F soit bijective.

9. Rejouer les démonstrations

```
# Laboratoire
python3 cryptovault/lab/attack_collision_birthday.py
python3 cryptovault/lab/benchmark_numtheory.py
python3 cryptovault/lab/attack_rsa_lab.py
python3 cryptovault/lab/attack_dh_mitm.py
python3 cryptovault/lab/attack_dh_bsgs.py
python3 cryptovault/lab/attack_ecc_nonce.py
python3 cryptovault/lab/attack_symmetric_lab.py
python3 cryptovault/lab/demo_pki_lab.py
python3 cryptovault/lab/attack_prng_lab.py

# Serveur
python3 cryptovault/app/server.py init --data-dir .server_data
python3 cryptovault/app/server.py status --data-dir .server_data

# Client
python3 cryptovault/app/client.py register --username alice --password
"SecretAlice123!" --keystore alice_ks.json
python3 cryptovault/app/client.py test-session --username alice --password
"SecretAlice123!" --keystore alice_ks.json --message "Message ultra-secret !"

# Coffre-fort
python3 cryptovault/app/vault.py add --file document.pdf --user alice --
keystore alice_ks.json --password "SecretAlice123!"
python3 cryptovault/app/vault.py list
python3 cryptovault/app/vault.py share --file document.pdf --user alice --
keystore alice_ks.json --password "SecretAlice123!" --recipient-user bob --
recipient-cert bob_cert.json
python3 cryptovault/app/vault.py get --file document.pdf --user bob --
keystore bob_ks.json --password "SecretBob123!" --out document_recupere.pdf
```

10. Conclusion

CryptoVault relie la théorie et la pratique : depuis l'algorithme d'Euclide étendu jusqu'au handshake de type TLS 1.3 et à l'arbre de Merkle, chaque couche a été construite, testée sur des vecteurs de référence et confrontée à ses attaques classiques.

Trois enseignements ressortent :

1. Les standards (RSA-OAEP, RFC 6979, RFC 3526, AES-GCM) offrent une sécurité solide **lorsqu'ils sont respectés à la lettre**.
2. Un seul oubli, comme un nonce réutilisé, un mode sans intégrité, un PRNG faible ou l'absence d'authentification, suffit à tout briser, comme l'ont montré les démonstrations du laboratoire.
3. La cryptographie asymétrique actuelle ne survivra pas à un ordinateur quantique de grande taille : la migration vers ML-KEM et ML-DSA, d'abord en mode hybride, doit être planifiée dès maintenant.

Ce prototype est un outil d'apprentissage et d'audit, pas un produit : en production, on s'appuie sur des bibliothèques auditées.